

Here is the implementation for **Stage 4: auth middleware and logout endpoint**. We extract the validation logic into a reusable guard and implement a protected logout route.

Node.js (Express & JavaScript)

javascript

```
import { createClient } from '@supabase/supabase-js';
import express from 'express';

const app = express();
app.use(express.json());

const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_ANON_KEY);

// =====
// 1. REUSABLE MIDDLEWARE GUARD
// =====
async function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: "Access token required" });
  }

  const token = authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({ error: "Access token required" });
  }

  const { data: { user }, error } = await supabase.auth.getUser(token);

  if (error || !user) {
    return res.status(401).json({ error: "Invalid or expired token" });
  }

  // Attach the verified user and token to the request object
  req.user = user;
  req.token = token;
  next();
}

// =====
```

```
// 2. PUBLIC & AUTH ROUTES (From Stage 1)
// =====
app.post('/auth/signup', async (req, res) => { /* ... */ });
app.post('/auth/login', async (req, res) => { /* ... */ });
app.get('/public/info', (req, res) => { /* ... */ });

// POST /auth/logout (Protected Route)
app.post('/auth/logout', requireAuth, async (req, res) => {
  // Pass the user's specific access token to scope the sign-out globally
  const { error } = await supabase.auth.admin.signOut(req.token);

  if (error) {
    return res.status(400).json({ error: error.message });
  }

  return res.sendStatus(204);
});

// =====
// 3. PROTECTED ROUTES
// =====
app.get('/protected/profile', requireAuth, (req, res) => {
  return res.status(200).json({
    id: req.user.id,
    email: req.user.email,
    created_at: req.user.created_at
  });
});

// CHECKPOINT: Second protected route using the exact same guard
app.get('/protected/dashboard', requireAuth, (req, res) => {
  return res.status(200).json({
    message: `Welcome to your dashboard, ${req.user.email}!`,
    stats: { premium: true }
  });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Use code with caution.

Python (FastAPI)

python

```
import os
from fastapi import FastAPI, HTTPException, status, Depends, Request
from fastapi.responses import JSONResponse
from supabase import create_client, Client

app = FastAPI()
supabase: Client = create_client(os.environ.get("SUPABASE_URL"), os.environ.get("SUPABASE_

# =====
# 1. REUSABLE DEPENDENCY GUARD
# =====
def get_current_user(request: Request):
    auth_header = request.headers.get("Authorization")

    if not auth_header or not auth_header.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Access token required")

    try:
        token = auth_header.split(" ")[1]
    except IndexError:
        raise HTTPException(status_code=401, detail="Access token required")

    try:
        response = supabase.auth.get_user(token)
        user = response.user
        if not user:
            raise ValueError()

        # Return a dictionary containing both user details and the raw token
        return {"user": user, "token": token}
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid or expired token")

# Custom global handler to force FastAPI's HTTPException detail into your custom {"error":
@app.exception_handler(HTTPException)
async def custom_http_exception_handler(request: Request, exc: HTTPException):
    return JSONResponse(status_code=exc.status_code, content={"error": exc.detail})

# =====
# 2. PUBLIC & AUTH ROUTES
```

```
# =====
@app.post("/auth/signup")
def signup(): pass

@app.post("/auth/login")
def login(): pass

@app.get("/public/info")
def public_info(): pass

# POST /auth/Logout (Protected Route)
@app.post("/auth/logout", status_code=status.HTTP_204_NO_CONTENT)
def logout(current_auth: dict = Depends(get_current_user)):
    try:
        # FastAPI's supabase client exposes sign_out. We supply the JWT scope if needed.
        supabase.auth.sign_out(current_auth["token"])
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
    return None

# =====
# 3. PROTECTED ROUTES
# =====
@app.get("/protected/profile")
def get_protected_profile(current_auth: dict = Depends(get_current_user)):
    user = current_auth["user"]
    return {
        "id": user.id,
        "email": user.email,
        "created_at": user.created_at
    }

# CHECKPOINT: Second protected route using the exact same guard
@app.get("/protected/dashboard")
def get_protected_dashboard(current_auth: dict = Depends(get_current_user)):
    user = current_auth["user"]
    return {
        "message": f"Welcome to your dashboard, {user.email}!",
        "stats": {"premium": True}
    }
```

Use code with caution.

Now that you have completed the base authentication loop, where should we go next?

- Implement **Role-Based Access Control (RBAC)** to restrict dashboard access to admin users
- Create a **PostgreSQL schema migration** script to initialize user profiles in Supabase
- Set up **automated testing** for your middleware and auth endpoints